Advanced C Programming Assignment Report

Task 1: Data Design Review

Task 1.1 - Diagnose the Design

Deficiencies identified in the primary health centre's patient register:

- **Parallel Arrays:** Maintaining related data across separate arrays risks synchronization bugs if indices mismatch during deletions/sorting.
- **Inadequate String Buffer for Phone:** `char phone[100][10]` allocates 10 bytes, leaving no room for the null terminator `\0` for a 10-digit number, causing buffer overflows.
- **Composite Value in Float:** Storing blood pressure (systolic/diastolic, e.g., 120/80) as a single float loses independent tracking and precision.
- **Fixed-Length Date Text:** `char visitDate[8]` cannot properly format standard `DD/MM/YYYY` strings (requires 10+ bytes).
- **Absence of Limit Check:** `count` lacks bounds checking in `addPatient()`, risking heap/stack corruption.

Task 1.2 & 1.3 - Repair & Design Defence

Design Defence (under 100 words): Struct-based design encapsulates related patient attributes into a single cohesive unit. This eliminates parallel array synchronization errors, simplifies sorting operations by swapping entire records in `O(1)` pointer/struct assignments, enables clean function parameter passing via pointers, and allows seamless future extension without modifying existing function signatures.

Task 2: Courier Parcel Tracking

Task 2.1 & 2.2 - Entities & Structure Definitions

Entities: `date` (day, month, year), `address` (city, pincode), `parcel` (id, weight, destination address, booking date).

```
struct date { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel { int id; float weight; struct address to; struct date booked; };
```

Task 2 Implementation: Courier Tracking Code & Terminal Output

```c
#include <stdio.h>

struct date { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel { int id; float weight; struct address to; struct date booked; };

void print_parcel(const struct parcel *p) {
    printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
           p->id, p->weight, p->to.city, p->to.pincode,
           p->booked.day, p->booked.month, p->booked.year);
}

void sort_by_weight(struct parcel p[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (p[j].weight < p[min_idx].weight) { min_idx = j; }
        }
        if (min_idx != i) {
            struct parcel temp = p[i];
            p[i] = p[min_idx];
            p[min_idx] = temp;
        }
    }
}

int main(void) {
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++) { print_parcel(&list[i]); }
    return 0;
}
```

Access Paths (Task 2.4)

- **Using array and dot operator:** `list[2].to.pincode`
- **Using pointer and arrow operator:** `p->to.pincode`

Program Execution Output

```
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
```

Task 3: Unions and Memory Layout (Weather Sensor Packets)

Task 3.1 & 3.2 - Structure vs. Union & Size Prediction

A tagged union is ideal for the weather station because packets carry *only one* data type at a time. A structure holding all fields simultaneously would waste memory.

Size Prediction (64-bit system): Enum tag consumes 4 bytes. The union contains a float (4 bytes), int (4 bytes), and `char[8]` (8 bytes), making the union size 8 bytes due to alignment padding. With alignment padding for the outer structure, total `sizeof(struct reading)` = 12 bytes.

Task 3.3 Implementation: Tagged Union Code

```c
#include <stdio.h>
enum kind { TEMPERATURE, HUMIDITY, STATUS };
struct reading {
    enum kind type;
    union {
        float celsius;
        int percent;
        char status[8];
    } value;
};
void show(const struct reading *r) {
    switch (r->type) {
        case TEMPERATURE: printf("Temp %.1f C\n", r->value.celsius); break;
        case HUMIDITY: printf("Humidity %d%%\n", r->value.percent); break;
        case STATUS: printf("Status %s\n", r->value.status); break;
    }
}
int main(void) {
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n", sizeof(log[0].valu
e), sizeof(log[0]));
    for (int i = 0; i < 3; i++) show(&log[i]);
    return 0;
}
```

Expected Output

```
sizeof union = 8, sizeof struct reading = 12
Temp 31.5 C
Humidity 68%
Status OK
```

Task 4: Structure Forensics (Bank Savings Account)

Task 4.1 & 4.2 - Locate Defects & Evidence

- **Defect 1 (Balances unchanged):** `addInterest()` accepted the structure *by value*, mutating a local copy instead of the original structure in `main()`.
- **Defect 2 (Compilation error):** `printf("%d %f\n", p.number, p.balance);` failed because `p` is a pointer (`struct account *p`), so it requires the arrow operator (`p->number`).

Task 4.3 - Repaired Program

```c
#include <stdio.h>
struct account { int number; float balance; };

void addInterest(struct account *a, float rate) {
    a->balance += a->balance * rate / 100.0f;
}

int main(void) {
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    struct account *p = list;
    for (int i = 0; i < 3; i++) {
        addInterest(&list[i], 4.0f);
        printf("Account %d: Balance = %.2f\n", (p + i)->number, (p + i)->balance);
    }
    return 0;
}
```

Task 4.4 - Passing Guidelines

- **By Value:** Use for small, primitive-like lightweight structures where copy overhead is negligible and side-effect isolation is desired.
- **By Pointer:** Use when functions must modify caller data or avoid large memory copy overhead.
- **By Const Pointer:** Use for large structures read-only, optimizing performance while preventing accidental mutations.

Task 5: Storage Classes (Hostel Mess Token Counter)

Task 5.1 & 5.2 - Four Implementations & Requirement Changes

Comparing automatic, static local, global extern, and register storage classes. Static local preserves state between function calls across invocations while restricting scope to the function block.

Task 5.3 Implementation: Static Counter Code

```c
#include <stdio.h>

int next_token(void) {
    static int token = 1000;
    return ++token;
}

int auto_token(void) {
    int token = 1000;
    return ++token;
}

int main(void) {
    int s1 = next_token(), s2 = next_token(), s3 = next_token();
    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();
    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto: %d %d %d\n", a1, a2, a3);
    return 0;
}
```

Expected Output

```
static: 1001 1002 1003
auto: 1001 1001 1001
```

Register Variable Note: Taking the address of a register variable (`&var`) is illegal because register variables may be stored directly in CPU registers rather than addressable RAM memory locations.

Task 6: Design Challenge (College Canteen Inventory & Billing)

Stage 1 & 2 - Problem Interpretation & Data Model

The canteen system manages packaged items (price, expiry date), cooked items (price, preparation time), and combo meals (up to 3 component items). Requirements include recording sales, stock updates, 2-day expiry warnings, and revenue-sorted billing summaries.

Data Model: Uses a tagged union to distinguish item types, nested structures for dates/times, and an array of structures to hold canteen inventory.

Stage 3 & 4 - Critical Design Decisions & Structures

- **Safe Stock Updates:** Validate that requested sale quantity is less than or equal to available stock prior to decrementing.
- **Expiry Checking:** Compare item expiry dates against current date structure fields (day, month, year) within a 48-hour threshold.

```
enum item_type { PACKAGED, COOKED, COMBO };
struct expiry_date { int day, month, year; };
struct prep_time { int hour, minute; };

struct canteen_item {
    int id;
    char name[30];
    float price;
    int stock;
    enum item_type type;
    union {
        struct expiry_date expiry;
        struct prep_time prepared;
        int combo_item_ids[3];
    } details;
};
```

Stage 5 - Implementation Summary

The completed program successfully processes simulated sales transactions, triggers out-of-stock validation guards, scans inventory for items nearing expiration within 2 days, and generates a formatted daily sales summary sorted in descending order of revenue.